**RedBlackTree.java BinaryNode.java BinarySearchTree.java RedBlackNode.java RedBlackTreeDriver.java RedBlaceTreeDriver2.java**

Create a red black tree class that will behave like red black tree. You will be given 2 drivers to test your red black tree on: RedBlackTreeDriver.java and RedBlackTreeDriver2. RedBlackTreeDriver will test insertion into a red black tree, while RedBlackTreeDriver2 will also test deletion. Do not alter the drivers in any way. The following is the RedBlackTreeDriver:

```
import java.util.*;
import static java.lang.System.*;
import java.io.*;

public class RedBlackTreeDriver {
	public static void main(String args[])throws IOException
	{
		Scanner input = new Scanner(System.in);

		RedBlackTree tree = new RedBlackTree();
		out.println("Enter new value. (-1 to stop): ");
		int num = input.nextInt();
		while(num!=-1)
		{
			tree.add(new RedBlackNode(num));
			num = input.nextInt();
		}
		tree.printFullTree(tree.fullLevelOrder(), 6);
	}
}
```

Take note that, whereas you have the freedom to create the class as you see fit, is a RedBlackTree method called printFullTree that requires the return from another method RedBlackTree method called fullLevelOrder and an integer. The method fullLevelOrder should return a List or Queue of the full level order of the resulting tree. The printFullTree would print the full tree up to the 6th level.

A sample of a run through RedBlackTreeDriver appears below:

```
Enter new value. (-1 to stop):
7 27 11 3 14 12 26 41 19 35 4 50 -1
14
11|27
4|12|26|41
3|7|--|--|19|--|35|50
--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--
--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--
```

When writing your red black tree class, you may choose to inherit your previous BinaryNode class and BinarySearchTree class.

The following is the RedBlacTreeDriver2 class:

```java
import java.util.*;
import static java.lang.System.*;
import java.io.*;

public class RedBlackTreeDriver2 {
	public static void main(String args[])throws IOException
	{
		Scanner input = new Scanner(System.in);

		RedBlackTree tree = new RedBlackTree();
		out.println("Enter new value. (-1 to stop): ");
		int num = input.nextInt();
		while(num!=-1)
		{
			tree.add(new RedBlackNode(num));
			num = input.nextInt();
		}

		out.println("Enter value to delete. (-1 to stop): ");
		num = input.nextInt();
		while(num!=-1)
		{
			tree.remove(num);
			num = input.nextInt();
		}
		tree.printFullTree(tree.fullLevelOrder(), 6);

}
```

The RedBlackTreeDriver2 class is an extension of the previous class, but this time it uses deletion as well. A sample of a run through RedBlackTreeDriver2 appears below:

```
Enter new value. (-1 to stop):
7 27 11 3 14 12 26 41 19 35 4 50 -1 26 14 50 41 27 35 -1
Enter value to delete. (-1 to stop):
11
4|19
3|7|12|--
--|--|--|--|--|--|--|--
--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--
--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--
```